

รายงานโครงงาน

OuterSpace

จัดทำโดย

นาย สรธรณ์ สุขสุโชติ รหัสนักศึกษา 6704062611298

เสนอ

ผู้ช่วยศาสตราจารย์ ดร. สถิต ประสมพันธ์

รายงานนี้เป็นส่วนหนึ่งของรายวิชา Object Oriented Programming

ภาควิชา วิทยาการคอมพิวเตอร์และสาระสนเทศ คณะ วิทยาศาสตร์ประยุกต์

มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ

ภาคเรียนที่ 1 / 2569

บทที่ 1
บทนำ

1.1 ที่มาและความสำคัญของโปรเจ็ค

เกมแนว Shoot 'em Up เป็นเกมที่ผู้เล่นควบคุมยานเพื่อยิงศัตรูและหลบกระสุนจำนวนมาก จุดเด่นคือการตอบสนองที่รวดเร็วและรูปแบบการโจมตีที่อ่านออกได้ ในขณะที่เกมแนว Roguelite เพิ่มความหลากหลายให้การเล่นแต่ละรอบ (Run) ด้วยการสุ่มเส้นทาง รางวัล และเหตุการณ์ ผู้เล่นต้องตัดสินใจเลือกเส้นทางและอัปเกรดเพื่อสร้างยานในแบบของตนเอง

โครงงาน **OuterSpace** จึงนำทั้งสองแนวมารวมกัน ผู้เล่นควบคุมยานอวกาศในหน้าจอแนวตั้งอัตราส่วน 9:16 (450 × 800 พิกเซล) ยิงศัตรูที่เข้ามาเป็นคลื่น (Wave) เลือกเส้นทางบนแผนที่ที่สร้างขึ้นจากค่า Seed เก็บทองและอัปเกรดยาน แล้วต่อสู้กับบอสท้ายแต่ละ Act ทั้งหมด 3 Act

ผู้จัดทำเลือกพัฒนาเกมด้วยภาษา Java ล้วน โดยใช้เพียง Java Standard Library (Swing และ Java 2D) ไม่ใช้ Game Engine หรือ Framework ภายนอก ทำให้ต้องออกแบบและเขียนเองทั้งหมด ตั้งแต่ Game Loop การรับอินพุต การวาดภาพ การตรวจสอบการชน ไปจนถึงอัลกอริทึมสร้างแผนที่ จึงเป็นโอกาสที่ดีในการประยุกต์ใช้หลักการเขียนโปรแกรมเชิงวัตถุ (OOP) กับระบบที่มีหลายส่วนทำงานร่วมกัน

1.2 ประเภทของโครงการ

ตารางที่ 1.1 ประเภทของโครงการ ภาษา และเทคโนโลยีที่ใช้

หัวข้อ	รายละเอียด
ประเภทของโปรแกรม	เกมคอมพิวเตอร์ แนวยิงยานอวกาศแนวตั้ง (Vertical Shoot 'em Up) ผสมระบบ Roguelite เล่นได้ 1 ผู้เล่น บนคอมพิวเตอร์ (Windows)
ภาษาที่ใช้	Java (คอมไพล์และทดสอบด้วย JDK 26.0.2)
เทคโนโลยีที่ใช้	Java Standard Library เท่านั้น ได้แก่ Java Swing (JFrame, JPanel, javax.swing.Timer) สำหรับหน้าต่างและการรับอินพุต, Java 2D (Graphics2D, BufferedImage) สำหรับการวาดภาพ, javax.imageio.ImageIO สำหรับโหลด Sprite และ Git สำหรับจัดการเวอร์ชัน
ข้อจำกัดที่ตั้งใจ	ไม่ใช้ Game Engine หรือ Framework ภายนอก เช่น Unity, Unreal, JavaFX, LibGDX โปรแกรมเริ่มทำงานเป็นแอปพลิเคชันบนเดสก์ท็อป (ไม่ใช่ Applet) รันผ่านไฟล์ run.bat

1.3 ประโยชน์

1. ได้ประยุกต์ใช้หลักการเขียนโปรแกรมเชิงวัตถุ ได้แก่ Encapsulation, Inheritance, Polymorphism, Abstraction และ Composition กับงานจริงที่มีหลายระบบทำงานร่วมกัน
2. ได้พัฒนาระบบหลักของเกมยิงยานอวกาศ ได้แก่ Game Loop การเคลื่อนที่ การยิง การตรวจสอบการชน ระบบศัตรู และระบบบอส โดยไม่พึ่งพา Game Engine
3. ได้ฝึกออกแบบโครงสร้างโปรแกรมให้แยกหน้าที่ชัดเจนเป็นแพ็กเกจ และขยายระบบได้ด้วยการเพิ่มข้อมูล (Data-driven)
4. ได้ฝึกใช้ Lambda และ Functional Interface (Supplier, Consumer, Predicate, Runnable) เก็บพฤติกรรมไว้ในข้อมูล
5. ได้ใช้ Git บันทึกและติดตามการเปลี่ยนแปลงของโค้ด และฝึกเขียนเอกสารออกแบบก่อนลงมือเขียนโค้ด

1.4 ขอบเขตของโครงการ

ขอบเขตของโครงการอ้างอิงจากเอกสารออกแบบ JAVA ในการเขียนโดยใช้หลักการเขียนโปรแกรมแบบ OOP

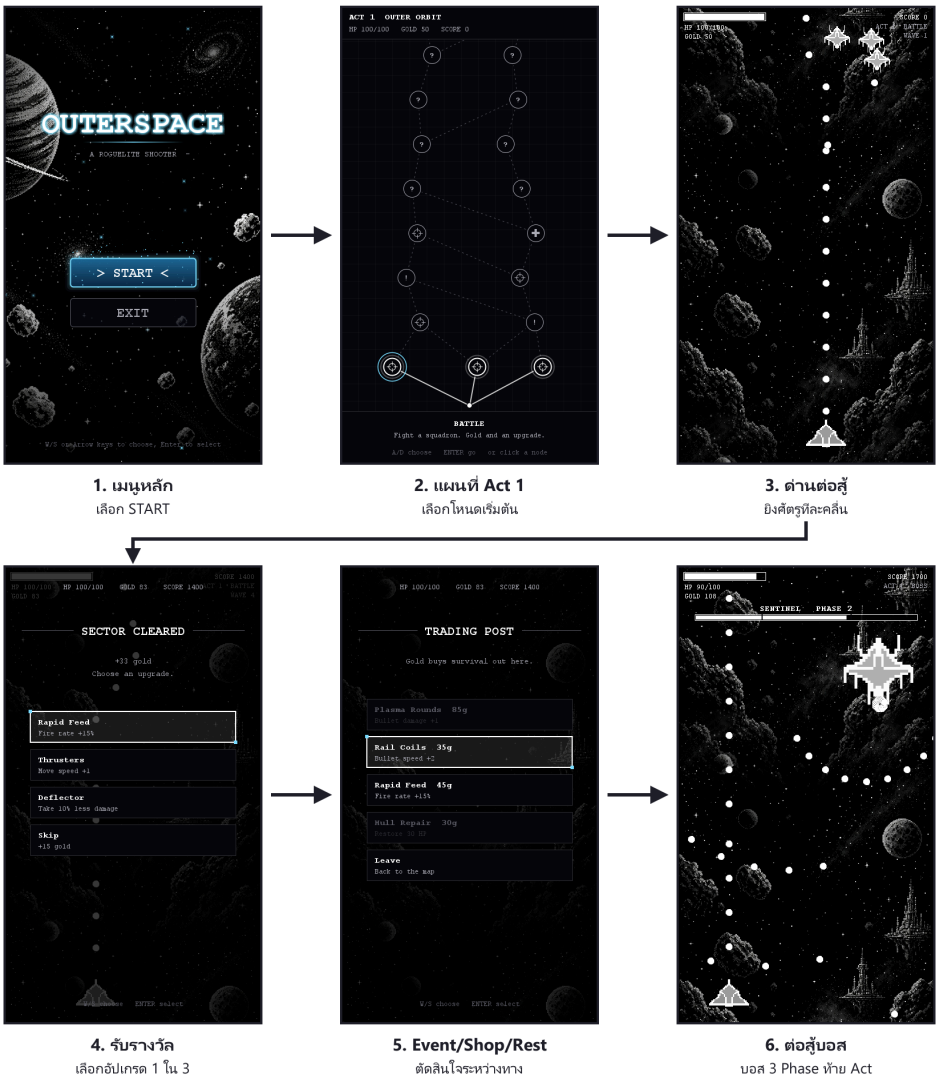
1.4.1 ฟังก์ชันของเกม

ตารางที่ 1.2 สถานะของฟีเจอร์ในโครงงาน

ระบบ	สถานะ	รายละเอียด / คลาสที่เกี่ยวข้อง
เมนูหลัก + พื้นหลังเคลื่อนไหว + อนุภาค	เสร็จแล้ว	MainMenu, MenuParticles
หน้าจอแนวตั้ง 9:16 + พื้นหลัง Parallax 3 ชั้น	เสร็จแล้ว	Constants, ParallaxBackground
การเคลื่อนที่และการยิงของยานผู้เล่น	เสร็จแล้ว	Player, CombatStage
ศัตรู 2 ชนิด (ยิง / พุ่งชน) และระบบคลื่น	เสร็จแล้ว	ShootingEnemy, RammingEnemy, WaveSpawner
บอส 3 ตัว แต่ละตัวมี 3 Phase	เสร็จแล้ว	Boss, BossData
แผนที่ Roguelite แบบโหนด 6 ชนิด สร้างจาก Seed	เสร็จแล้ว	MapGenerator, MapConfig, PathManager
3 Act ความยากเพิ่มขึ้นตามข้อมูล	เสร็จแล้ว	ActData.ACTS
ทอง คะแนน อัปเกรด 7 แบบ รางวัลหลังด่าน	เสร็จแล้ว	RunState, Upgrade, RewardManager
ร้านค้า อีเวนต์ 6 แบบ จุดพัก	เสร็จแล้ว	Shop, EventLibrary, RewardManager.rest
ระบบทักษะ อาวุธหลายชนิด บันทึกข้อมูลถาวร	วางแผนไว้	ยังไม่ได้พัฒนาตาม GAME DESIGN

1.4.2 Storyboard

รูปที่ 1.1 แสดงลำดับการเล่นหลักของเกม ภาพทั้งหมดเป็นภาพหน้าจอจริงที่ได้จากการรันเกม (Seed 1234) ผู้เล่นเริ่มจากเมนูหลัก เลือกโหนดบนแผนที่ เข้าสู่ด่านต่อสู้ รับรางวัล และต่อสู้กับบอสท้าย Act



1. เมนูหลัก
เลือก START

2. แผนที่ Act 1
เลือกโหนดเริ่มต้น

3. ด่านต่อสู้
ยิงศัตรูทีละคลื่น

4. รับรางวัล
เลือกอัปเกรด 1 ใน 3

5. Event/Shop/Rest
ตัดสินใจระหว่างทาง

6. ต่อสู้บอส
บอส 3 Phase ท้าย Act

รูปที่ 1.1 Storyboard ลำดับการเล่นของ OuterSpace (ภาพหน้าจอจริง)

บทที่ 2
การพัฒนา

2.1 เนื้อเรื่องย่อและวิธีการเล่น

OuterSpace เป็นเกมยิงยานอวกาศแนวตั้งผสม Roguelite การเล่นหนึ่งรอบประกอบด้วย 3 Act ได้แก่ Act 1 Outer Orbit, Act 2 Ruined Spires และ Act 3 The Core แต่ละ Act มีแผนที่ของตนเอง ผู้เล่นเดินทางจากด้านล่างของแผนที่ขึ้นไปหาบอสด้านบน เป้าหมายคือเอาชนะบอสทั้ง 3 ตัวโดยที่ HP ไม่ลดลงเหลือ 0 HP คงอยู่ต่อเนื่องระหว่างด่าน และรีเซ็ตเฉพาะเมื่อเริ่มรอบใหม่เท่านั้น ศัตรูเข้ามาเป็นคลื่น 2 ชนิดสลับกัน ได้แก่ ศัตรูยิง (ShootingEnemy) ที่ลาดตระเวนแล้วยิงเล็งผู้เล่น และศัตรูพุ่งชน (RammingEnemy) ที่พุ่งเข้าหาผู้เล่น ส่วนบอสแต่ละตัวมี 3 Phase ยิงถี่ขึ้นและเพิ่มรูปแบบกระสุนเมื่อ HP ลดลง

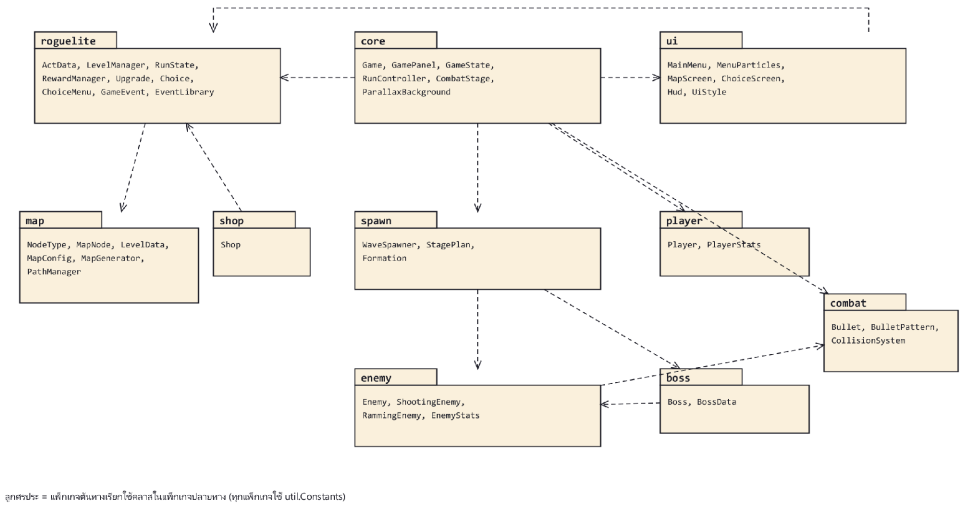
ตารางที่ 2.1 ปุ่มควบคุม (GamePanel.handleActionKey และ tick)

หน้าจอ	ปุ่ม / เมาส์	การทำงาน
เมนูหลัก	W/S หรือ ↑/↓, เมาส์ชี้	เลือก START หรือ EXIT
เมนูหลัก	Enter / Space / คลิก	ยืนยันตัวเลือก
แผนที่	A/D, W/S หรือลูกศร, เมาส์ชี้	เลือกโหนดที่ไปได้ (Enter / Space / คลิก = เข้าโหนด)
ด่านต่อสู้	W A S D หรือลูกศร (กดค้าง)	เคลื่อนที่ยาน 8 ทิศทาง
ด่านต่อสู้	Space (กดค้าง)	ยิงกระสุนต่อเนื่องตามอัตรายิง
หน้าตัวเลือก	W/S หรือ A/D, เมาส์ชี้	เลือกอัปเกรด / อีเวนต์ / สินค้า (Enter / Space / คลิก = ยืนยัน)
ทุกหน้าจอ	Esc	ออกจากเกม

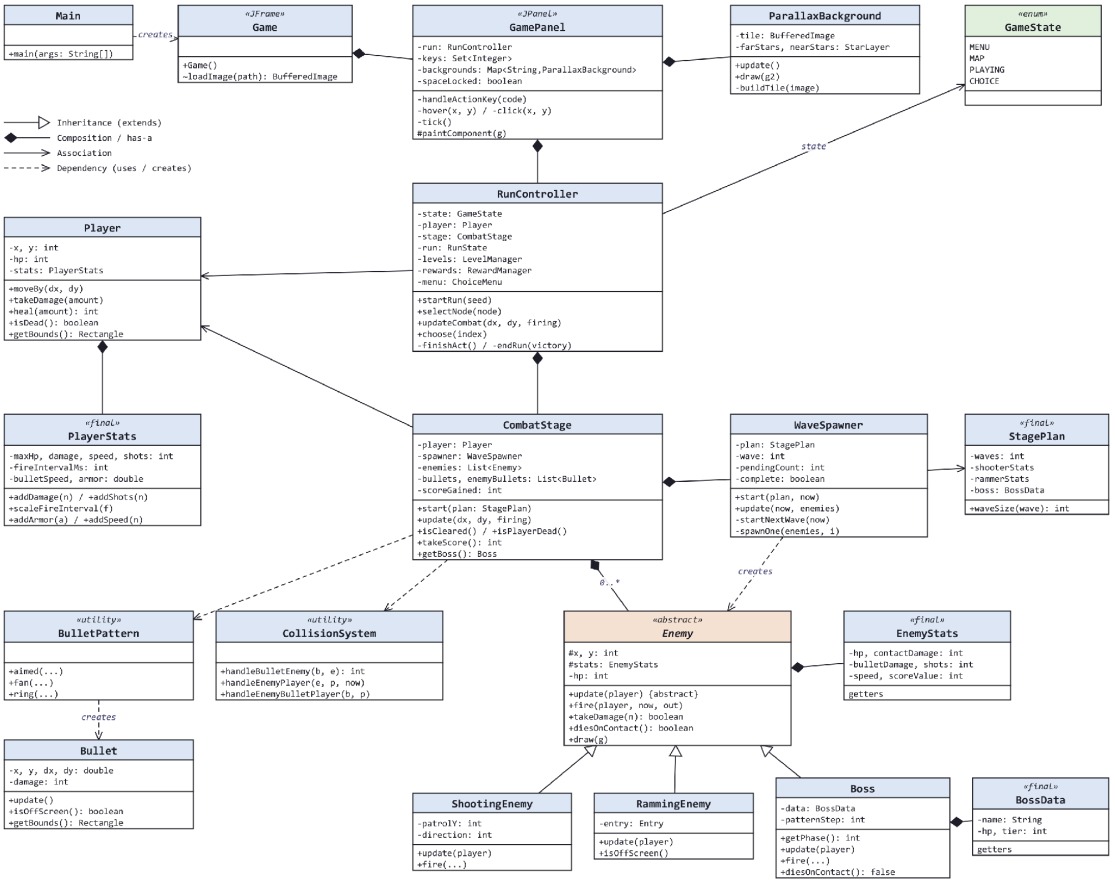
1. **เริ่มเกม** เลือก START ระบบสร้าง Seed และแผนที่ Act 1 แล้วแสดงแถบชื่อ Act ประมาณ 2.2 วินาที
2. **เลือกเส้นทาง** โหนดที่เลือกได้จะสว่างและกะพริบ มีโหนด 6 ชนิด ได้แก่ Battle, Elite, Event, Shop, Rest และ Boss
3. **ต่อสู้** กด Space ค้างเพื่อยิง หลบกระสุนและศัตรูที่พุ่งชน คลื่นคี่เป็นศัตรูยิง คลื่นคู่เป็นศัตรูพุ่งชน
4. **เก็บทรัพยากร** เคลียร์ด่านแล้วได้ทองและเลือกอัปเกรด 1 จาก 3 หรือ Skip รับ 15 ทอง
5. **ตัดสินใจระหว่างทาง** ที่ Shop ซื้อสินค้า ที่ Event เลือกแลกความเสี่ยงกับรางวัล ที่ Rest ซ่อมยานหรือรับอัปเกรดฟรี
6. **ชนะ/แพ้** ชนะบอส Act 3 แสดง VICTORY ส่วน HP เหลือ 0 แสดง SHIP DESTROYED พร้อมคะแนนและ Seed ที่เมนูหลัก

2.2 คลาสไดอะแกรม พร้อมคำอธิบาย

โครงการมีซอร์สโค้ด 44 ไฟล์ (รวม 4,807 บรรทัด) แบ่งเป็น 12 แพ็กเกจ รูปที่ 2.1 แสดงความสัมพันธ์ระหว่างแพ็กเกจ รูปที่ 2.2 แสดงคลาสส่วนแกนกลาง การต่อสู้ ศัตรู และบอส และรูปที่ 2.3 แสดงคลาสส่วน Roguelite แผนที่ ร้านค้า และอีเวนต์ แผนภาพทั้งหมดสร้างจากซอร์สโค้ดจริง โดยแสดงเฉพาะ Field และ Method ที่สำคัญ

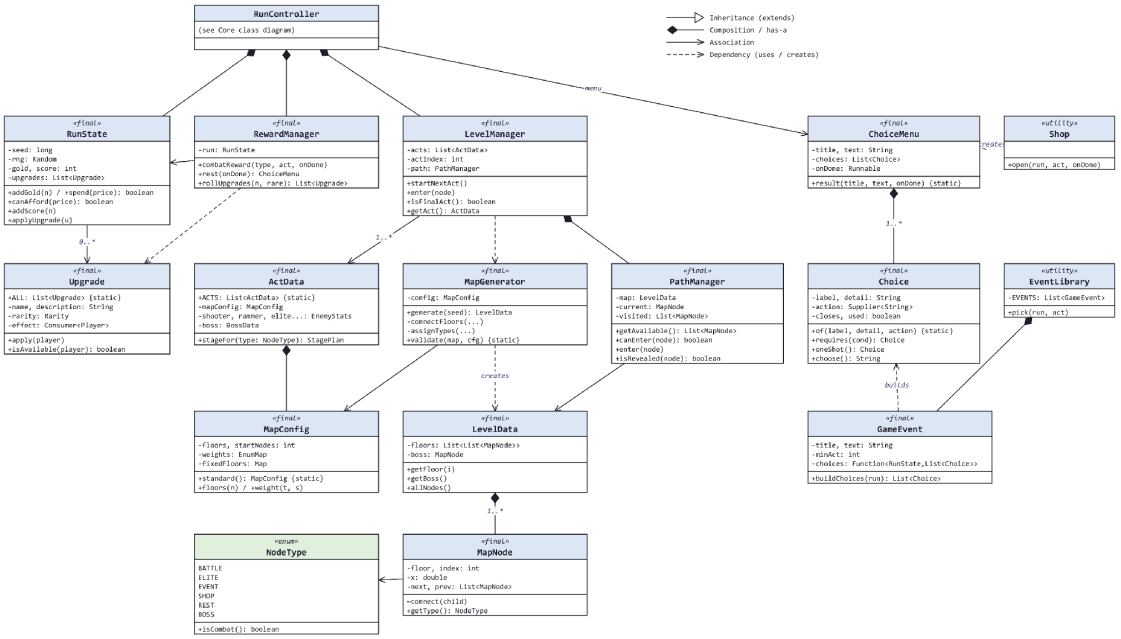


รูปที่ 2.1 ความสัมพันธ์ระหว่างแพ็กเกจของโครงการ



รูปที่ 2.2 Class Diagram ส่วน Core, Player, Combat, Spawn, Enemy และ Boss

ความสัมพันธ์หลักในรูปที่ 2.2 ได้แก่ `ShootingEnemy`, `RammingEnemy` และ `Boss` สืบทอดจากคลาสนามธรรม `Enemy` ส่วน `Game` → `GamePanel` → `RunController` → `CombatStage` → `WaveSpawner` เป็นการประกอบกันแบบ Composition โดยแต่ละชั้นสร้างและถือครองชั้นถัดไป `CollisionSystem` และ `BulletPattern` เป็นคลาสยูทิลิตีที่มีเฉพาะเมธอด static



รูปที่ 2.3 Class Diagram ส่วน Roguelite, Map, Shop และ Event

ในรูปที่ 2.3 `RunController` ถือครอง `RunState` (ข้อมูลของรอบปัจจุบัน), `LevelManager` (ลำดับ Act และแผนที่) และ `RewardManager` (รางวัล) ทุกหน้าจอที่ให้ผู้เล่นเลือกถูกแทนด้วยวัตถุ `ChoiceMenu` ที่มี `Choice` หลายตัว ทำให้ใช้หน้าจอ `ChoiceScreen` และโค้ดรับอินพุตชุดเดียวกันได้ทั้งหมด

ตารางที่ 2.2 หน้าที่ของคลาสหลักในแต่ละแพ็กเกจ

แพ็กเกจ	คลาส	หน้าที่
outerspace	Main	จุดเริ่มโปรแกรม สร้าง Game บน Event Dispatch Thread
core	Game, GamePanel, GameState, RunController, CombatStage, ParallaxBackground	หน้าต่างเกม Game Loop (Timer 60 FPS) สถานะของเกม ลำดับการเล่นทั้งรอบ ด่านต่อสู้ และพื้นหลังเลื่อน
player	Player, PlayerStats	ตำแหน่ง HP การรับดาเมจ และค่าพลังที่อัปเกรดได้
combat	Bullet, BulletPattern, CollisionSystem	กระสุน รูปแบบการยิง และการตรวจการชน
enemy / boss	Enemy, ShootingEnemy, RammingEnemy, Boss, EnemyStats, BossData	ศัตรู บอส และค่าพลังของศัตรูและบอส
spawn	WaveSpawner, StagePlan, Formation	การเกิดศัตรูเป็นคลื่นและแผนของด่าน
map	MapNode, LevelData, MapConfig, MapGenerator, PathManager, NodeType	โครงสร้างแผนที่ การสร้างและตรวจสอบแผนที่ และตำแหน่งผู้เล่น
roguelite	ActData, LevelManager, RunState, RewardManager, Upgrade, Choice, ChoiceMenu, GameEvent, EventLibrary	ข้อมูล Act เงิน คะแนน อัปเกรด รางวัล และอีเวนต์
shop / ui / util	Shop, MainMenu, MapScreen, ChoiceScreen, Hud, UiStyle, Constants	ร้านค้า หน้าจอทั้งหมด (อ่านข้อมูลอย่างเดียว) และค่าคงที่ส่วนกลาง

2.3 รูปแบบการพัฒนา Application

โครงงานนี้พัฒนาแบบเพิ่มทีละส่วน (Iterative and Incremental Development) เริ่มจากต้นแบบที่เล่นได้ขนาดเล็ก แล้วเพิ่มระบบทีละชั้น โดยแต่ละรอบมีการวางแผน ออกแบบ เขียนโค้ด ทดสอบ และแก้ไข ตามลำดับ commit ใน Git รายละเอียดแต่ละขั้นตอนมีดังนี้

1. **Planning** กำหนดแนวเกม (Shoot 'em Up + Roguelite) และข้อจำกัด: ใช้ Java ล้วน ไม่ใช้ Game Engine เขียนเอกสาร GAME DESIGN.md ระบุ Core Gameplay Loop ค่าพลัง ศัตรู บอส แผนที่ และเศรษฐกิจในเกม

2. **Analyze / Design** แยกระบบออกเป็นแพ็กเกจและเขียน ARCHITECTURE กำหนดลำดับการทำงานในแต่ละเฟรม และกฎการพึ่งพาระหว่างระบบ (UI อ่านข้อมูลอย่างเดียว MapGenerator ไม่ยุ่งกับการต่อสู้)
3. **Implementation รอบที่ 1–2** ยานผู้เล่น กระสุน การชน และศัตรู 2 ชนิดพร้อมระบบคลื่น
4. **Implementation รอบที่ 3** เมนูหลัก หน้าจอ 9:16 และพื้นหลัง Parallax
5. **Implementation รอบที่ 4** ระบบ Roguelite ทั้งหมดและบอส 3 Phase พร้อมแยก CombatStage และ RunController ออกจาก GamePanel
6. **Testing และแก้ไข** คอมไพล์และรันผ่าน run.bat หลังการเปลี่ยนแปลงสำคัญ ทดสอบด้วยการรันเกมจริงด้วย Seed คงที่และทดสอบหน่วย 19 กรณี (ผ่านทั้งหมด)

โปรแกรมเริ่มทำงานจากเมธอด `Main.main` ซึ่งเรียก `SwingUtilities.invokeLater` เพื่อสร้างวัตถุ `Game` บน Event Dispatch Thread ตามข้อกำหนดของ Swing `Game` (สืบทอด `JFrame`) โหลดภาพและสร้าง `GamePanel` ส่วน `GamePanel` (สืบทอด `JPanel`) สร้าง `javax.swing.Timer` ที่เรียก `tick()` ทุก 1000/60 มิลลิวินาที ทำให้เกมวนลูปอัปเดตและวาดภาพที่ประมาณ 60 FPS จนกว่าผู้เล่นจะปิดโปรแกรม

2.4 อธิบายส่วนของโปรแกรมที่มี (หลักการ OOP)

2.4.1 Constructor

Constructor ใช้กำหนดค่าเริ่มต้นของวัตถุ คลาสลูกใช้ `super(...)` เรียก Constructor ของคลาสแม่ก่อน แล้วจึงกำหนดค่าเฉพาะของตนเอง เช่น `ShootingEnemy` กำหนดแถวลาดตระเวนและสุ่มเวลายิงนัดแรก เพื่อไม่ให้ศัตรูทั้งคลื่นยิงพร้อมกัน นอกจากนี้ `Choice` ใช้ `private` Constructor ร่วมกับ Static Factory Method และคลาสยูทิลิตีอย่าง `CollisionSystem` ใช้ `private` Constructor เพื่อป้องกันการสร้างวัตถุ

enemy/ShootingEnemy.java

```
public ShootingEnemy(int x, int patrolY, BufferedImage image, BufferedImage bulletImage, EnemyStats
stats) {
    super(x, 0, image, stats);
    this.bulletImage = bulletImage;
    this.patrolY = patrolY;
    this.y = -displayHeight; // enter from above the top edge
    // Stagger the first volley so a wave doesn't fire in unison.
    this.lastShotTime = System.currentTimeMillis() - (long) (Math.random() * stats.getFireIntervalMs()
/ 2);
}
```

2.4.2 Encapsulation

ข้อมูลภายในคลาสประกาศเป็น `private` และเปิดให้ภายนอกเข้าถึงผ่านเมธอดที่ควบคุมความถูกต้องเท่านั้น ตัวอย่างที่ชัดคือ `RunState` ซึ่งเป็นระบบเศรษฐกิจของเกม ทองเปลี่ยนได้ผ่าน `addGold` (ไม่ติดลบ) และ `spend` (ไม่ยอมให้จ่ายเกินที่มี) เท่านั้น ร้านค้าและอีเวนต์จึงแก้ค่าทองโดยตรงไม่ได้ ส่วน `Player.takeDamage` คำนวณการลดดาเมจจากเกราะภายในคลาส ผู้เรียกจึงไม่ต้องรู้สูตร และเมธอดอย่าง `MapNode.getNext()` คืน `Collections.unmodifiableList(...)` เพื่อป้องกันการแก้ไขโครงสร้างแผนที่

roguelite/RunState.java

```
public void addGold(int amount) {
    gold = Math.max(0, gold + amount);
}

public boolean canAfford(int price) {
    return gold >= price;
}

/** Spends gold if there is enough. Returns {@code false} and changes nothing otherwise. */
public boolean spend(int price) {
    if (!canAfford(price)) {
        return false;
    }
```

```
    gold -= price;
    return true;
}
```

player/Player.java
```
public void takeDamage(int amount) {
    int reduced = Math.max(1, (int) Math.round(amount * (1 - stats.getArmor())));
    hp = Math.max(0, hp - reduced);
}
```

2.4.3 Composition

โครงงานใช้ Composition เป็นหลักแทนการสืบทอดหลายชั้น วัตถุแต่ละตัวถือครองวัตถุที่รับผิดชอบงานย่อย เช่น `RunController` ถือครองส่วนประกอบของการเล่นหนึ่งรอบทั้งหมด และ `GamePanel` สร้างหน้าจอและตัวควบคุมการเล่น

core/RunController.java – Field ของคลาส
```
private final Player player;
private final CombatStage stage;
private GameState state = GameState.MENU;
private RunState run;
private LevelManager levels;
private RewardManager rewards;
private ChoiceMenu menu;
```

ตารางที่ 2.3 ความสัมพันธ์แบบ Composition ที่สำคัญ

คลาสเจ้าของ	ถือครอง	ความหมาย
GamePanel	RunController, MainMenu, MapScreen, ChoiceScreen, Hud, ParallaxBackground	หน้าจอแต่ละแบบและตัวควบคุมการเล่น
RunController	CombatStage, RunState, LevelManager, RewardManager, ChoiceMenu	ส่วนประกอบของการเล่นหนึ่งรอบ
CombatStage	Player, WaveSpawner, List<Enemy>, List<Bullet> × 2	ทุกสิ่งในด่านต่อสู้
ActData	MapConfig, EnemyStats × 4, BossData	ข้อมูลทั้งหมดของหนึ่ง Act
ChoiceMenu	List<Choice>, Runnable onDone	หน้าจอตัวเลือก และสิ่งที่ทำต่อเมื่อเลือกเสร็จ

2.4.4 Polymorphism

`CombatStage` เก็บศัตรูทุกชนิดไว้ใน `List<Enemy>` รายการเดียว และเรียก `update`, `fire`, `isOffScreen` ผ่านชนิด `Enemy` โดยไม่ต้องตรวจว่าเป็นศัตรูชนิดใด เมธอดที่ทำงานจริงขึ้นอยู่กับชนิดของวัตถุขณะรัน (Dynamic Binding) ทำให้ `CollisionSystem` จัดการได้ทั้งศัตรูปกติที่ถูกทำลายเมื่อชนและบอสที่ไม่ถูกทำลาย ด้วยการถามวัตถุผ่าน `diesOnContact()` และ `tryContactHit(now)` แทนการตรวจชนิดด้วย `instanceof`

core/CombatStage.java – วนอัปเดตศัตรูทุกชนิดผ่านชนิด Enemy
```
Iterator<Enemy> enemyIt = enemies.iterator();
while (enemyIt.hasNext()) {
    Enemy enemy = enemyIt.next();
    enemy.update(player);
    if (enemy.isOffScreen()) {
        enemyIt.remove();
        continue;
    }
    enemy.fire(player, now, enemyBullets);
}
```

Polymorphism อีกรูปแบบคือการเก็บพฤติกรรมไว้ใน Lambda เช่น `Upgrade` แต่ละตัวเก็บผลไว้ใน `Consumer<Player>` เมื่อเรียก `upgrade.apply(player)` จะทำงานต่างกันตามอัปเกรดแต่ละตัว

roguelite/Upgrade.java

```
new Upgrade("Rapid Feed", "Fire rate +15%", Rarity.COMMON, 45,
        p -> p.getStats().scaleFireInterval(0.85),
        p -> p.getStats().getFireIntervalMs() > PlayerStats.MIN_FIRE_INTERVAL_MS),
    ...

public void apply(Player player) {
    effect.accept(player);
}
```

2.4.5 Abstract

Enemy เป็น Abstract Class ที่ประกาศ `public abstract void update(Player player);` บังคับให้คลาสลูกทุกตัวต้องกำหนดการเคลื่อนที่ของตนเอง และไม่สามารถสร้างวัตถุ Enemy โดยตรงได้ โครงงานไม่ได้สร้าง Interface ของตนเอง แต่ใช้ Interface ของ Java เช่น `KeyListener`, `MouseAdapter`, `Runnable`, `Consumer<Player>` และ `Predicate<Player>`

enemy/Enemy.java

```
/** Per-frame movement. Subclasses decide how to chase or patrol. */
public abstract void update(Player player);

/**
 * Adds any bullets fired this frame to {@code out}. The base class never
 * fires; only shooting enemies override this.
 */
public void fire(Player player, long now, List<Bullet> out) {
    // no weapon
}
```

2.4.6 Inheritance

ShootingEnemy, RammingEnemy และ Boss สืบทอด (extends) จาก Enemy ได้รับ Field และเมธอดร่วม ได้แก่ ตำแหน่ง ขนาด HP การรับดาเมจ การวาดภาพแบบกลับหัว และขอบเขตสำหรับตรวจการชน คลาสลูกเพิ่มเฉพาะพฤติกรรมที่ต่างกัน ส่วนฝั่ง UI ใช้การสืบทอดจากคลาสของ Swing ได้แก่ `Game extends JFrame` และ `GamePanel extends JPanel` ซึ่ง Override `paintComponent` เพื่อวาดเกม

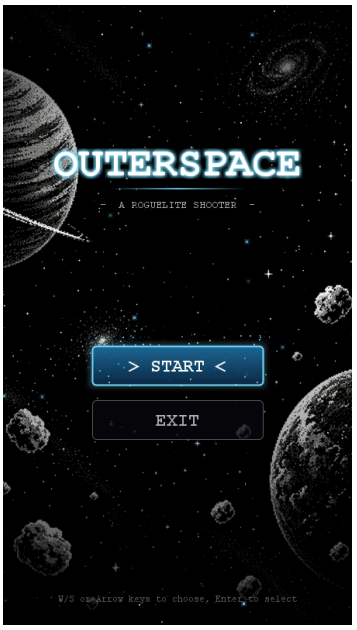
การประกาศคลาสที่ใช้การสืบทอด

```
public abstract class Enemy { ... }
public class ShootingEnemy extends Enemy { ... }
public class RammingEnemy extends Enemy { ... }
public class Boss extends Enemy { ... }

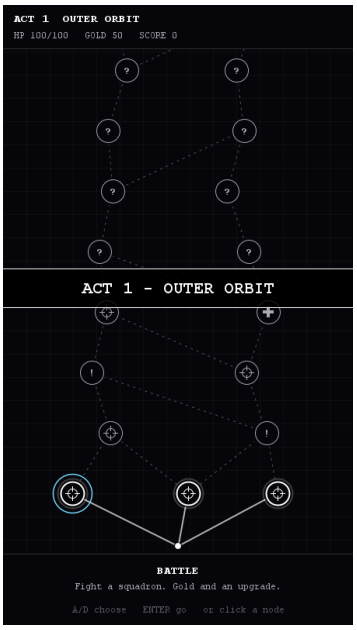
public class Game extends JFrame { ... }
public class GamePanel extends JPanel { ... }
```

2.5 หน้าจอ GUI

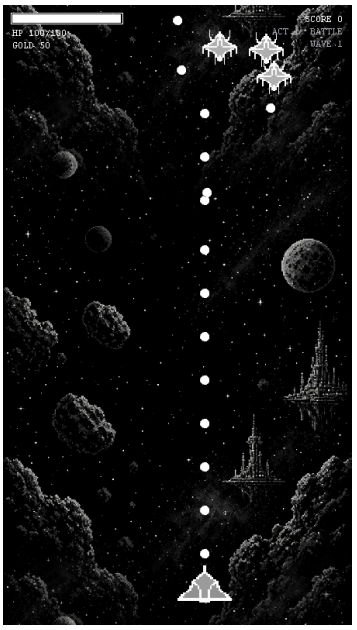
ทุกหน้าจอวาดด้วย Java 2D บน `GamePanel` เพียงแผงเดียว (ไม่ได้ใช้ Component สำเร็จรูปอย่าง `JButton`) ปุ่มและตัวเลือกเป็นสี่เหลี่ยมที่วาดเองและตรวจตำแหน่งเมาส์ด้วย `Rectangle.contains` สีและฟอนต์ที่ใช้ร่วมกันกำหนดไว้ใน `UiStyle` (โทนขาวดำ ฟอนต์ Monospaced และสีเน้นฟ้า) โครงสร้างของ GUI คือ `Game` (JFrame) บรรจุ `GamePanel` (JPanel) เพียงตัวเดียว แล้ว `GamePanel` เลือกวาดหน้าจอตามสถานะ `GameState` ได้แก่ MENU, MAP, PLAYING และ CHOICE โดยแต่ละหน้าจอมีคลาสรับผิดชอบการวาดของตนเอง



(ก) เมนูหลัก

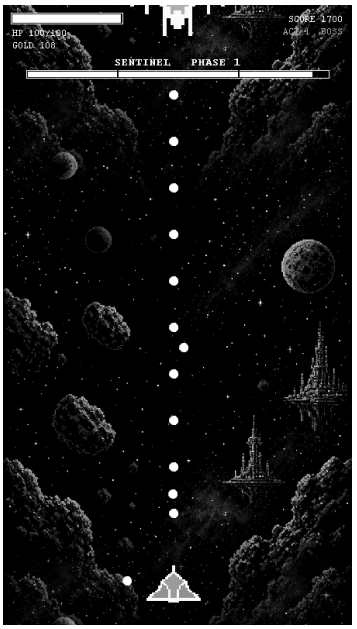


(ข) แผนที่ + แถบชื่อ Act

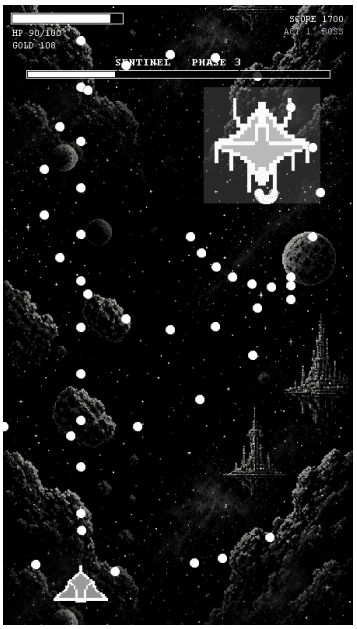


(ค) ด่านต่อสู้ (ศัตรูยิง)

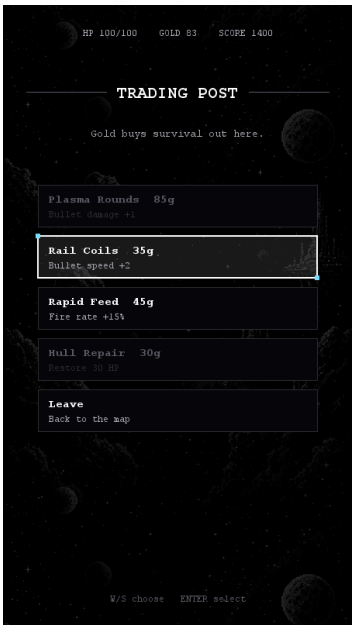
รูปที่ 2.4 หน้าจอหลักของเกม



(ก) บอส Phase 1



(ข) บอส Phase 3



(ค) ร้านค้า (ChoiceScreen)

รูปที่ 2.5 การต่อสู้กับบอสและหน้าตัวเลือก

ตารางที่ 2.4 องค์ประกอบของแต่ละหน้าจอ

หน้าจอ (คลาส)	องค์ประกอบสำคัญ
เมนูหลัก (MainMenu)	พื้นหลังภาพ + ไล่เฉด, อนุภาคฝุ่นดาวและดาวตก (MenuParticles), ชื่อเกมเรืองแสง, สรุปผลรอบที่แล้ว (VICTORY / SHIP DESTROYED, คะแนน, Seed), ปุ่ม START และ EXIT
แผนที่ (MapScreen)	แถบบน (ชื่อ Act, HP, ทอง, คะแนน), โหนด 6 ชนิดมีไอคอนต่างกัน, สถานะโหนด (ผ่านแล้ว/เลือกได้/ถูกตัดขาด), เส้นเชื่อม, แถบล่างแสดงชื่อและคำอธิบายโหนดที่เลือก
การเล่น + HUD (Hud)	หลอด HP และทอง (ซ้ายบน), คะแนน ชื่อ Act หมายเลขคลื่น (ขวาบน), หลอดเลือดบอสพร้อมเส้นแบ่ง Phase ที่ 70% และ 30%, พื้นหลัง Parallax, กระสุน ศัตรู และยานผู้เล่น
หน้าตัวเลือก (ChoiceScreen)	ใช้หน้าเดียวกันทั้งรางวัล ร้านค้า อีเวนต์ และจุดพัก: แถบสถานะ ชื่อหน้าจอ ข้อความอธิบาย และปุ่มตัวเลือก (ปุ่มที่เลือกไม่ได้แสดงสีจาง สินค้าที่ซื้อแล้วแสดง SOLD)

2.6 อธิบาย Event handling

GamePanel ลงทะเบียน KeyListener แบบ Anonymous Class การรับอินพุตแบ่งเป็น 2 แบบ: (1) **ปุ่มที่กดค้าง** สำหรับการเคลื่อนที่และการยิง เก็บใน Set<Integer> keys แล้ว Game Loop อ่านทุกเฟรม ทำให้กดหลายปุ่มพร้อมกันได้ (เช่น เคลื่อนที่ทแยงพร้อมยิง) และ (2) **ปุ่มคำสั่งครั้งเดียว** เช่น เลือกเมนูหรือยืนยัน จัดการทันทีใน handleActionKey ตามสถานะปัจจุบัน

core/GamePanel.java

```
addKeyListener(new KeyListener() {
    @Override
    public void keyTyped(KeyEvent e) {
        // unused
    }

    @Override
    public void keyPressed(KeyEvent e) {
        keys.add(e.getKeyCode());
        handleActionKey(e.getKeyCode());
    }

    @Override
    public void keyReleased(KeyEvent e) {
        keys.remove(e.getKeyCode());
        if (e.getKeyCode() == KeyEvent.VK_SPACE) {
            spaceLocked = false;
        }
    }
});
```

core/GamePanel.java

```
private void handleActionKey(int code) {
    if (code == KeyEvent.VK_ESCAPE) {
        System.exit(0);
    }
    boolean up = code == KeyEvent.VK_W || code == KeyEvent.VK_UP;
    boolean down = code == KeyEvent.VK_S || code == KeyEvent.VK_DOWN;
    boolean left = code == KeyEvent.VK_A || code == KeyEvent.VK_LEFT;
    boolean right = code == KeyEvent.VK_D || code == KeyEvent.VK_RIGHT;
    boolean confirm = code == KeyEvent.VK_ENTER || (code == KeyEvent.VK_SPACE && !spaceLocked);

    switch (run.getState()) {
        case MENU:
            if (up) {
                menu.moveUp();
            } else if (down) {
                menu.moveDown();
            } else if (confirm) {
                selectMenuOption();
            }
            break;
        case MAP:
            PathManager path = run.getLevels().getPath();
            if (left || up) {
                mapScreen.moveSelection(path, -1);
            } else if (right || down) {
                mapScreen.moveSelection(path, 1);
            } else if (confirm) {
                enterNode(mapScreen.getTarget(path));
            }
            break;
        case CHOICE:
            if (up || left) {
                choiceScreen.moveSelection(run.getMenu(), -1);
            } else if (down || right) {
                choiceScreen.moveSelection(run.getMenu(), 1);
            } else if (confirm) {
                choose(choiceScreen.getSelected(run.getMenu()));
            }
            break;
        default:
            break;
    }
```

```
}
```

สำหรับเมาส์ `MouseAdapter` ตัวเดียวลงทะเบียนเป็นทั้ง `MouseListener` และ `MouseMotionListener` การขยับเมาส์เรียก `hover(x, y)` เพื่อไฮไลต์ปุ่มหรือโหนดใต้เมาส์ และการคลิกเรียก `click(x, y)` ซึ่งยืนยันตัวเลือกนั้นตามสถานะปัจจุบัน นอกจากนี้ปุ่ม Space ใช้ทั้งยิงและยืนยัน เมื่อด่านจบขณะกด Space ค้าง การกดซ้ำอัตโนมัติ (Key repeat) อาจเลือกรางวัลโดยไม่ตั้งใจ จึงตั้ง `spaceLocked = true` เมื่อด่านจบและปลดล็อกเมื่อปล่อย Space

ตารางที่ 2.5 เหตุการณ์สำคัญและผลที่เกิดขึ้น

เหตุการณ์	เมธอดที่ตรวจ / จัดการ	ผล
เลือก START	GamePanel.selectMenuOption → RunController.startRun	รีเซ็ตยาน ทอง 50 สร้างแผนที่ Act 1
เข้าโหนดต่อสู้	RunController.selectNode	CombatStage.start(plan), สถานะ PLAYING
เข้าโหนด Shop/Rest/Event	RunController.selectNode	สร้าง ChoiceMenu, สถานะ CHOICE
กระสุนโดนศัตรู	CollisionSystem.handleBulletEnemy	ลด HP ศัตรู ลบกระสุน ศัตรูตายได้คะแนน
ศัตรู/กระสุนโดนผู้เล่น	handleEnemyPlayer / handleEnemyBulletPlayer	Player.takeDamage (หักเกราะ)
บอส HP ต่ำกว่า 70% / 30%	Boss.getPhase	ยิงถี่ขึ้น เพิ่มรูปแบบกระสุน
เคลียร์ด่าน	WaveSpawner.isComplete → RewardManager.combatReward	ได้ทอง หน้าเลือกอัปเกรด
ชนะบอส	RunController.finishAct	Act ถัดไป หรือ VICTORY ถ้าเป็น Act 3
HP เป็น 0	RunController.updateCombat / choose	endRun(false) กลับเมนูพร้อมสรุปผล

2.7 อธิบายอัลกอริทึมที่สำคัญในโปรแกรม

2.7.1 Game Loop

`javax.swing.Timer` เรียก `tick()` ทุก 1000/60 ms (ประมาณ 60 FPS) บน Event Dispatch Thread แต่ละเฟรมอัปเดตข้อมูลก่อน แล้วจึงเรียก `repaint()` ให้วาดใหม่

Pseudocode – CombatStage.update

```
ALGORITHM CombatStage.update(dx, dy, firing)
    stats ← player.getStats()
    player.moveBy(dx × stats.speed, dy × stats.speed)       // clamp ภายในจอ
    now ← เวลาปัจจุบัน (ms)
    IF firing AND now − lastShotTime ≥ stats.fireIntervalMs THEN
        BulletPattern.fan(bullets, ..., มุม −90°, stats.shots, 8°, stats.bulletSpeed, stats.damage)
        lastShotTime ← now
    อัปเดตกระสุนผู้เล่น และลบกระสุนที่ออกนอกจอ
    spawner.update(now, enemies)
    FOR EACH enemy IN enemies
        enemy.update(player)                                 // polymorphic
        IF enemy.isOffScreen() THEN ลบ enemy; CONTINUE
        enemy.fire(player, now, enemyBullets)                // polymorphic
    อัปเดตกระสุนศัตรู และลบกระสุนที่ออกนอกจอ
    scoreGained += CollisionSystem.handleBulletEnemy(bullets, enemies)
    CollisionSystem.handleEnemyPlayer(enemies, player, now)
    CollisionSystem.handleEnemyBulletPlayer(enemyBullets, player)
```

2.7.2 รูปแบบการยิงกระสุน

Pseudocode – BulletPattern

```
ALGORITHM BulletPattern.fan(out, sx, sy, angle, count, spreadDeg, speed, damage)
    step  ← toRadians(spreadDeg)
    start ← angle − step × (count − 1) / 2          // ให้กระสุนกระจายสมมาตรรอบ angle
    FOR i ← 0 TO count − 1
        a ← start + i × step
```

```
        out.add(new Bullet(sx, sy, cos(a) × speed, sin(a) × speed, damage))

ALGORITHM BulletPattern.aimed(... sx, sy, targetX, targetY ...)
    base ← atan2(targetY − sy, targetX − sx)         // มุมจากผู้ยิงไปยังเป้าหมาย
    fan(out, sx, sy, base, count, spreadDeg, speed, damage)
```

การยิงได้เมื่อเวลาผ่านไปไม่น้อยกว่า `fireIntervalMs` นับจากนัดก่อน กระสุนแต่ละนัดเก็บเวกเตอร์ความเร็ว `(dx, dy)` เป็น double จึงเคลื่อนที่ในแนวทแยงได้อย่างราบรื่น

2.7.3 การตรวจสอบการชน

ใช้การตรวจสอบสี่เหลี่ยมซ้อนทับ (Axis-Aligned Bounding Box) ผ่าน `Rectangle.intersects` `CollisionSystem` ตรวจ 3 กรณี ได้แก่ กระสุนผู้เล่นกับศัตรู ศัตรูกับผู้เล่น และกระสุนศัตรูกับผู้เล่น โดยใช้ `Iterator.remove()` ลบวัตถุระหว่างวนลูปได้อย่างปลอดภัย ความซับซ้อนคือ O(B × E) โดย B คือจำนวนกระสุน และ E คือจำนวนศัตรู ซึ่งเพียงพอสำหรับจำนวนวัตถุบนจอ และเมื่อกระสุนโดนศัตรูตัวแรกจะ `break` ออกจากลูป กระสุนหนึ่งนัดจึงทำดาเมจได้เพียงครั้งเดียว

combat/CollisionSystem.java

```
public static int handleBulletEnemy(List<Bullet> bullets, List<Enemy> enemies) {
    int scoreGained = 0;

    Iterator<Bullet> bulletIt = bullets.iterator();
    while (bulletIt.hasNext()) {
        Bullet bullet = bulletIt.next();
        Rectangle bulletBounds = bullet.getBounds();

        Iterator<Enemy> enemyIt = enemies.iterator();
        while (enemyIt.hasNext()) {
            Enemy enemy = enemyIt.next();
            if (bulletBounds.intersects(enemy.getBounds())) {
                bulletIt.remove();
                if (enemy.takeDamage(bullet.getDamage())) {
                    enemyIt.remove();
                    scoreGained += enemy.getScoreValue();
                }
                break;
            }
        }
    }
    return scoreGained;
}
```

2.7.4 การสร้างแผนที่

แต่ละ Act มีแผนที่เป็นกราฟหลายชั้น (Layered graph) สร้างโดย `MapGenerator` จาก Seed ของรอบการเล่น ดังนั้น Seed เดียวกันให้แผนที่เดิมเสมอ หัวใจของการสร้างเส้นเชื่อมคือ Monotone walk ใน `connectFloors` ตัวชี้ i (ชั้นล่าง) และ j (ชั้นบน) เริ่มจากโหนดซ้ายสุดและขยับไปทางขวาเท่านั้น แต่ละก้าวเลือกขยับทั้งคู่ (ตรงขึ้น) หรือขยับเพียงตัวเดียว (แตกแขนง / รวมทาง) แล้วเชื่อม lower[i] กับ upper[j] จึงรับประกันได้ว่าทุกโหนดมีเส้นเข้าและออก และเส้นไม่ตัดกัน หลังสร้างเสร็จตรวจสอบความถูกต้องด้วย `validate()` (ความสามารถเข้าถึงได้ จำนวนเส้นทาง กฎการวางโหนด) ถ้าไม่ผ่านให้สร้างใหม่ด้วย Seed ถัดไปสูงสุด 50 ครั้ง

map/MapGenerator.java

```
private void connectFloors(List<MapNode> lower, List<MapNode> upper, Random rng) {
    int i = 0;
    int j = 0;
    lower.get(0).connect(upper.get(0));
    while (i < lower.size() - 1 || j < upper.size() - 1) {
        if (i == lower.size() - 1) {
            j++;
        } else if (j == upper.size() - 1) {
            i++;
        } else if (rng.nextDouble() < config.getStraightChance()) {
            i++;
            j++;
```

```
        } else if (rng.nextBoolean()) {
            i++;
        } else {
            j++;
        }
        lower.get(i).connect(upper.get(j));
    }

    for (MapNode from : lower) {
        for (MapNode to : upper) {
            if (from.getNext().contains(to)
                    || from.getNext().size() >= MAX_OUT_EDGES
                    || Math.abs(from.getX() - to.getX()) > MAX_LINK_DISTANCE
                    || crossesExisting(lower, from.getIndex(), to.getIndex())) {
                continue;
            }
            if (rng.nextDouble() < config.getExtraEdgeChance()) {
                from.connect(to);
            }
        }
    }
}
```

บทที่ 3
สรุปผลและข้อเสนอแนะ

โครงงาน OuterSpace พัฒนาเกมยิงยานอวกาศแนวตั้งผสม Roguelite ได้สำเร็จตามขอบเขตหลัก เกมเล่นได้ครบวงจรตั้งแต่เมนูหลัก แผนที่ 3 Act ด่านต่อสู้ ร้านค้า อีเวนต์ จุดพัก บอส จนถึงการสรุปผล พัฒนาด้วย Java Standard Library ล้วน ประกอบด้วย 44 ไฟล์ 12 แพ็กเกจ และนำหลักการเขียนโปรแกรมเชิงวัตถุมาใช้จริงทุกส่วน

3.1 ปัญหาที่พบระหว่างการพัฒนา

ตารางที่ 3.1 ปัญหาที่พบระหว่างพัฒนาและแนวทางแก้ไข

ปัญหา	สาเหตุ	แนวทางแก้ไข (หลักฐาน)
คอมไพล์ผ่าน run.bat ไม่สำเร็จหลังเพิ่มแพ็กเกจ spawn	คำสั่ง javac ระบุแพ็กเกจทีละโฟลเดอร์ จึงไม่รวมโฟลเดอร์ใหม่	เพิ่ม src/outerspace/spawn/*.java ใน run.bat (commit 00d6f56)
GamePanel มีหน้าที่มากเกินไป (เดิมรวมการต่อสู้ทั้งหมด)	เมื่อเพิ่มแผนที่และ Roguelite โค้ดในคลาสเดียวจะยาวและแก้ยาก	แยก logic การต่อสู้ไป CombatStage และลำดับการเล่นไป RunController (commit 5d11b8c)
กด Space ค้างตอนด่านจบแล้วเลือกรางวัลเอง	Space ใช้ทั้งยิงและยืนยัน และระบบปฏิบัติการส่ง Key repeat	เพิ่ม spaceLocked และล้าง keys เมื่อเข้าโหนดหรือเลือกตัวเลือก (GamePanel)
เห็นรอยต่อเมื่อพื้นหลังเลื่อนวนซ้ำ	ขอบบนและขอบล่างของภาพไม่ต่อเนื่องกัน	สร้างภาพต่อแบบ Crossfade ส่วนบน 25% กับส่วนล่างของภาพ (ParallaxBackground.buildTile)
ทุก Act ใช้พื้นหลังเดียวกัน	เดิมมีค่าคงที่ BACKGROUND_IMAGE ค่าเดียว	แยกเป็น ACT1–ACT3_BACKGROUND และย้ายภาพไป Map_assets (commit e385566)
แผนที่สุ่มอาจได้รูปแบบที่ไม่ดี เช่น เส้นทางน้อย หรือเส้นทางมีแต่ Battle	การสุ่มล้วนไม่รับประกันคุณภาพ	เพิ่ม validate() และสร้างใหม่ด้วย Seed ถัดไปสูงสุด 50 ครั้ง (MapGenerator)
ศัตรูทั้งคลื่นยิงพร้อมกัน	ทุกตัวเริ่มนับเวลายิงจากจุดเดียวกัน	สุ่มเวลาเริ่มของนัดแรก (ShootingEnemy Constructor)

หมายเหตุ: ปัญหาข้างต้นสรุปจากประวัติ commit และคอมเมนต์ในซอร์สโค้ด ปัญหาอื่นที่ผู้จัดทำพบระหว่างพัฒนา แต่ไม่มีบันทึกไว้ [ต้องเพิ่มเติมข้อมูล]

3.2 จุดเด่นของโปรแกรมที่ไม่เหมือนใคร

1. **พัฒนาด้วย Java ล้วน** ไม่ใช้ Game Engine เขียน Game Loop การวาดภาพ การชน และ UI เองทั้งหมด
2. **แผนที่สุ่มที่ตรวจสอบได้และทำซ้ำได้** Seed เดียวกันให้แผนที่เดิมทุกครั้ง และผ่านการตรวจสอบความถูกต้องทุกแผนที่ ทำให้รายงานผลการทดสอบและทบทวนเหตุการณ์ได้จาก Seed เดิม
3. **ขยายระบบด้วยข้อมูล** เพิ่ม Act อัปเกรด อีเวนต์ หรือบอสได้โดยเพิ่มข้อมูลในรายการเดียว ไม่ต้องแก้ Logic เช่น เพิ่ม Act ใน `ActData.ACTS`, เพิ่มอัปเกรดใน `Upgrade.ALL`, เพิ่มอีเวนต์ใน `EventLibrary.EVENTS`
4. **บอสที่มีความต่างกันจริง** บอส 3 ตัวมี 3 Phase และ Tier ต่างกัน Tier สูงเพิ่มกระสุนเกลียวและวงแหวนชั้นที่สอง ทำให้บอสแต่ละ Act เล่นไม่ซ้ำกัน
5. **การตัดสินใจที่มีความหมาย** อีเวนต์ทุกแบบแลกความเสี่ยงกับรางวัล และอัปเกรด 7 แบบทำให้แต่ละรอบสร้างยานได้ต่างกัน
6. **โครงสร้างแยกหน้าที่ชัดเจน** UI อ่านข้อมูลอย่างเดียว RunController ไม่ขึ้นกับ Swing และ CombatStage ไม่รู้จักปุ่มคีย์บอร์ด

3.3 คำแนะนำสำหรับผู้สอนและรุ่นน้องต่อไป

-